



API Reference

POMSKI

Live-Coding Music Environment

Write Python patterns in the browser and hear them play as MIDI in real time. Complete reference for PatternBuilder, LiveBridge, DataFeeds, harmony engine, song form, and the Ableton Live bridge.

`localhost:8080 REPL port 5555 MIDI ch 0-indexed`

Contents

01 Setup & REPL

02 Composition

Pattern registration · Playback control · Tweaks · Harmony · Song form · Scheduling

03 PatternBuilder (p)

Note placement · Rhythmic generators · Transformations · Timing & feel · Utilities

04 LiveBridge (live)

Transport · Clips & scenes · Mixer · Devices · Watch · ClyphX Pro · Raw OSC

05 DataFeeds (feeds)

06 Duration Constants (dur)

07 GM Drum Map

08 REPL Quick Reference

09 Ports & MIDI Channels

POMSKI API Reference

POMSKI is a live-coding music environment. Write Python patterns in the browser at `http://localhost:8080` and they play back as MIDI in real time.

Table of Contents

1. [Setup & REPL](#)
2. [Composition](#)
3. [PatternBuilder \(p\)](#)
4. [Note placement](#)
5. [Rhythmic generators](#)
6. [Transformations](#)
7. [music21 scales \(quantize_m21, microtuning\)](#)
8. [Timing & feel](#)
9. [Utilities](#)
10. [LiveBridge \(live\)](#)
11. [DataFeeds \(feeds\)](#)
12. [Duration constants \(dur\)](#)
13. [GM Drum Map](#)
14. [REPL Quick Reference](#)
15. [Ports](#)

Setup & REPL

POMSKI starts from `pomski_template.py`. Once running:

- Browser UI: `http://localhost:8080`
- REPL port: `5555` (plain TCP, newline-terminated code blocks)
- Quick command box in the UI: type code and press Enter to exec

The following objects are pre-wired into the REPL namespace:

Name	Type	Description
<code>composition</code>	<code>Composition</code>	Global controller (clock, patterns, harmony)

<code>live</code>	<code>LiveBridge</code>	Ableton Live bridge via AbletonOSC
<code>feeds</code>	<code>DataFeeds</code>	HTTP polling feeds that write into <code>composition.data</code>
<code>pat</code>	shorthand	Alias for <code>composition.pattern</code> — write <code>@pat(channel)</code> instead of <code>@composition.pattern(channel=..., length=4)</code>

Composition

`composition` is the top-level controller. It manages the clock, harmony, song structure, and all MIDI patterns.

Attributes

Attribute	Type	Description
<code>composition.bpm</code>	<code>float</code>	Current tempo
<code>composition.key</code>	<code>str</code>	None <code>Root key</code> (e.g. "C", "F#", "Bb")
<code>composition.data</code>	<code>dict</code>	Shared live data store — patterns read; feeds/OSC write
<code>composition.running_patterns</code>	<code>dict[str, Pattern]</code>	Active patterns keyed by name
<code>composition.sequencer</code>	<code>Sequencer</code>	Underlying beat clock
<code>composition.conductor</code>	<code>Conductor</code>	Time-varying signal engine
<code>composition._is_live</code>	<code>bool</code>	True after <code>composition.live()</code> is called
<code>composition._main_loop</code>	<code>asyncio.EventLoop</code>	The asyncio loop

Pattern registration

```
@composition.pattern(channel=0, length=4) def ch1(p): p.note(60, beat=0)
```

`composition.pattern(channel, length=4, unit=None, drum_note_map=None, voice_leading=False)`

Parameter	Description
-----------	-------------

<code>channel</code>	MIDI channel, 0-indexed (0–15). Channel 9 = GM drums.
<code>length</code>	Pattern length in beats (default 4). When <code>unit</code> is set, this is a note count.
<code>unit</code>	Duration of one step in beats (e.g. <code>dur.SIXTEENTH</code>). Sets <code>length</code> as a count and makes it the grid size. The total beat length (<code>length × unit</code>) must be > 1 beat (the default reschedule lookahead). Minimum useful unit is <code>0.25</code> (<code>dur.SIXTEENTH</code>).
<code>drum_note_map</code>	Dict of <code>{"name": midi_note}</code> . Pass <code>GM_DRUM_MAP</code> for standard GM names.
<code>voice_leading</code>	If <code>True</code> , chord inversions minimize voice movement automatically.

pat shorthand: `pat` is pre-loaded as an alias for `composition.pattern`. Both forms are equivalent:

```
# Full form @composition.pattern(channel=0, length=4) def ch1(p): p.note(60, beat=0) # Short form –
channel is positional, length defaults to 4 @pat(0) def ch1(p): p.note(60, beat=0) @pat(0, 8) #
channel 0, 8-beat pattern def ch1(p): p.note(60, beat=0)
```

Hot-swap: Redefine any function with the same name while running to replace it on the next cycle.

```
# Auto-assign: new name → first empty slot @composition.pattern(channel=0, length=4) def melody(p):
p.note(60, beat=0) # Hot-swap: same name → replaces in place @composition.pattern(channel=0,
length=4) def melody(p): p.note(67, beat=0)
```

Playback control

```
composition.mute("ch1") # Silence a pattern (keeps cycling) composition.unmute("ch1") # Resume
composition.set_bpm(128.0) # Change tempo immediately composition.live_info() # Returns dict: bpm,
key, bar, section, chord, patterns, data
```

Tweaks — live parameter injection

```
# Inside a pattern, declare a tweakable parameter: @composition.pattern(channel=0, length=4) def
bass(p): pitches = p.param("pitches", [48, 52, 55]) p.sequence(steps=[0, 4, 8, 12], pitches=pitches)
# From the REPL, override it: composition.tweak("bass", pitches=[36, 40, 43]) # Remove:
composition.clear_tweak("bass", "pitches") # remove one param composition.clear_tweak("bass") #
remove all tweaks # Inspect: composition.get_tweaks("bass") # returns current tweaks dict
```

Harmony

Requires `key` to be set at construction. The harmony engine uses a weighted chord transition graph to generate progressions.

```
composition.harmony( style="functional_major", # harmony style – see list below cycle_beats=4, #
beats per chord change gravity=1.0, # 0.0 = wander freely, 1.0 = stay near tonic nir_strength=0.5, #
melodic inertia (Narmour model) dominant_7th=True, # include V7 chords )
```

Harmony styles:

Style	Character
<code>functional_major</code> / <code>diatonic_major</code>	Standard major key progressions
<code>turnaround</code>	Jazz turnaround patterns
<code>aeolian_minor</code>	Natural minor
<code>dorian_minor</code>	Minor with raised 6th
<code>phrygian_minor</code>	Minor with flattened 2nd
<code>lydian_major</code>	Major with raised 4th
<code>mixolydian</code>	Major with flattened 7th
<code>chromatic_mediant</code>	Chromatic third relationships
<code>suspended</code>	Sus2/Sus4 dominant movement
<code>whole_tone</code>	Whole tone scale movement
<code>diminished</code>	Diminished chord movement

Using the current chord in a pattern:

```
@composition.pattern(channel=1, length=4, voice_leading=True) def pads(p, chord): p.chord(chord,
root=60, velocity=80, sustain=True)
```

The `chord` parameter is injected automatically when the function signature includes it.

Song structure (form)

```
composition.form([ ("intro", 4), ("verse", 8), ("chorus", 8), ("bridge", 4), ("chorus", 16), ]) # Use
section name inside patterns: @composition.pattern(channel=1, length=4) def melody(p): if p.section
and p.section.name == "chorus": p.note(72, beat=0) else: p.note(67, beat=0)
```

`p.section` attributes: `.name`, `.bar` (bar within section), `.bars` (total bars), `.progress` (0.0–1.0)

Scheduling

```
# Run a function every 4 beats composition.schedule(my_fn, cycle_beats=4) # Run once at startup
(populates composition.data before patterns build), then every 16 beats composition.schedule(my_fn,
cycle_beats=16, wait_for_initial=True)
```

PatternBuilder (p)

`p` is passed to every pattern function. It is the composer's palette — use it to place notes and apply transformations.

Context attributes

Attribute	Type	Description
<code>p.cycle</code>	<code>int</code>	How many times this pattern has rebuilt (0-indexed)
<code>p.bar</code>	<code>int</code>	Global bar counter
<code>p.grid</code>	<code>int</code>	Grid slots in this pattern (e.g. 16 for a 4-beat pattern)
<code>p.section</code>	<code>SectionInfo</code> <code> </code> <code>None</code>	Current form section
<code>p.rng</code>	<code>random.Random</code>	Seeded RNG (deterministic if <code>composition.seed</code> is set)
<code>p.data</code>	<code>dict</code>	Alias for <code>composition.data</code>
<code>p.conductor</code>	<code>Conductor</code>	Time-varying signal source (alias: <code>p.c</code>)

Note placement

`p.note(pitch, beat=0, velocity=100, duration=0.25)`

Place a single MIDI note.

```
p.note(60, beat=0) # Middle C on beat 1 p.note(60, beat=0, velocity=110, duration=0.5)
p.note("kick_1", beat=0) # Drum name (requires drum_note_map) p.note(67, beat=-0.5) # Negative = from
end of pattern
```

Parameter	Description
<code>pitch</code>	MIDI note number (0–127) or drum name string
<code>beat</code>	Beat position (0.0 = bar start). Negative values wrap from end.
<code>velocity</code>	MIDI velocity (0–127, default 100)
<code>duration</code>	Note length in beats (default 0.25 = 16th note)

`p.hit(pitch, beats, velocity=100, duration=0.1)`

Place the same pitch at multiple beat positions.

```
p.hit("snare_1", [1, 3]) # standard backbeat
```

`p.hit_steps(pitch, steps, velocity=100, duration=0.1, grid=None, probability=1.0)`

Place hits at grid step indices (0-indexed).

```
p.hit_steps("kick_1", [0, 8]) # beats 1 and 3 p.hit_steps("hi_hat_closed", range(16)) # every 16th
note p.hit_steps("hi_hat_closed", range(16), probability=0.8) # 80% chance each
```

Default grid = `p.grid` (16 for a 4-beat pattern with 16th-note resolution).

`p.sequence(steps, pitches, velocities=100, durations=0.1, grid=None, probability=1.0)`

Multi-parameter step sequencer. Steps through a list of pitches as it fires.

```
p.sequence(steps=[0, 4, 8, 12], pitches=[60, 62, 64, 65]) p.sequence(steps=[0, 4, 8, 12], pitches=60)
# same pitch all steps p.sequence(steps=[0, 4, 8, 12], pitches=[60, 62, 64, 65], velocities=[100, 80,
100, 80], durations=0.25)
```

p.seq(notation, pitch=None, velocity=100)

Mini-notation string sequencer. Events are distributed evenly across the pattern length.

```
p.seq("kick_1 . [kick_1 kick_1] .") # kick with 16th subdivision
p.seq("60 [62 64] 67 60") # melody
p.seq(". snare_1?0.5 . snare_1") # 50% probability ghost note
p.seq("x x x x", pitch=60) # `x` = trigger using `pitch` param
```

Mini-notation syntax:

Token	Meaning
name or 60	Trigger note (pitch name or MIDI number)
. or ~	Rest
_	Sustain — extend the previous note
[a b c]	Group — subdivide one step into equal parts
x?0.6	Probability suffix — fires with given probability (0.0–1.0)

p.fill(pitch, step, velocity=100, duration=0.25)

Repeat a note at a fixed beat interval across the whole pattern.

```
p.fill(42, step=0.25) # 16th note hi-hats
p.fill(60, step=0.5) # 8th notes
```

p.arpeggio(pitches, step=0.25, velocity=85, duration=None, direction="up")

Cycle through a list of pitches at regular intervals.

```
p.arpeggio([60, 64, 67], step=0.25) # ascending 16th notes
p.arpeggio([60, 64, 67], step=0.25, direction="down") # descending
p.arpeggio([60, 64, 67], step=0.25, direction="up_down") # ping-pong
p.arpeggio([60, 64, 67], step=0.25, direction="random") # shuffle each cycle
```

p.chord(chord_obj, root, velocity=90, sustain=False, duration=1.0, inversion=0, count=None, legato=None)

Place a chord at beat 0. Requires `chord` injected via the pattern signature.

```
@composition.pattern(channel=1, length=4) def pads(p, chord): p.chord(chord, root=60, velocity=80, sustain=True) p.chord(chord, root=60, count=4, legato=0.9) # 4-note arpeggio feel
```

```
p.strum(chord_obj, root, velocity=90, offset=0.05, direction="up", ...)
```

Chord with staggered note onsets (guitar strum effect).

```
p.strum(chord, root=52, velocity=85, offset=0.06, legato=0.95) p.strum(chord, root=52, direction="down", offset=0.03)
```

Rhythmic generators

These are called once to generate a full rhythm for the pattern.

```
p.euclidean(pitch, pulses, velocity=100, duration=0.1, dropout=0.0, no_overlap=False)
```

Distribute `pulses` as evenly as possible across the pattern (Bjorklund/Euclidean rhythm). Produces many of the world's most common rhythms.

```
p.euclidean("kick_1", pulses=3) # 3-against-16 tresillo p.euclidean("snare_1", pulses=5, dropout=0.2) p.euclidean("hi_hat_closed", pulses=7, no_overlap=True)
```

`no_overlap=True` skips steps where the same pitch already exists — useful for layering ghost notes around hand-placed anchors.

```
p.bresenham(pitch, pulses, velocity=100, duration=0.1, dropout=0.0, no_overlap=False)
```

Alternative even distribution via the Bresenham line algorithm. Similar to Euclidean, often yields different accent patterns.

```
p.bresenham("snare_1", pulses=5)
```

```
p.bresenham_poly(parts, velocity=100, duration=0.1, grid=None, dropout=0.0, no_overlap=False)
```

Interlocking multi-voice rhythm generator. Voices never overlap — each step belongs to exactly one voice. Weights control density.

```
p.bresenham_poly( parts={"kick_1": 0.25, "snare_1": 0.125, "hi_hat_closed": 0.5}, velocity={"kick_1": 100, "snare_1": 90, "hi_hat_closed": 70}, )
```

`parts` dict: `{pitch: density_weight}`. Weights 0.0–1.0; remainder becomes rests. Use per-voice velocity dict or a single int for all voices.

Transformations

Call these **after** placing notes. They modify whatever is already in the pattern.

`p.transpose(semitones)`

```
p.transpose(12) # up one octave p.transpose(-5) # down a perfect fourth
```

`p.quantize(key, mode="ionian")`

Snap all pitches to the nearest scale degree.

```
p.quantize("C", "dorian") p.quantize("F#", "aeolian")
```

Scale modes: `ionian` (major), `dorian`, `phrygian`, `lydian`, `mixolydian`, `aeolian` (natural minor), `locrian`, `harmonic_minor`, `melodic_minor`

`p.quantize_m21(key, scale_name="MajorScale", scala_name=None)`

Snap all pitches to the nearest degree of any music21 scale class or Scala archive file.

```
p.quantize_m21("C", "MelodicMinorScale") p.quantize_m21("D", "HarmonicMinorScale")
p.quantize_m21("F#", "DorianScale") p.quantize_m21("Bb", "OctatonicScale") # Scala archive – takes
precedence over scale_name when given p.quantize_m21("C", scala_name="mbira_banda.scl")
p.quantize_m21("C", scala_name="pyth_12.scl")
```

Parameter	Description
<code>key</code>	Root note as a string: <code>"C"</code> , <code>"F#"</code> , <code>"Bb"</code> , etc.
<code>scale_name</code>	music21 scale class name (see table below). Default <code>"MajorScale"</code> . Ignored when <code>scala_name</code> is given.

<code>scala_name</code>	Filename from the Scala archive, e.g. <code>"mbira_banda.scl"</code> . When provided, overrides <code>scale_name</code> . The archive contains 3,935 tuning files covering historical temperaments, world music scales, and microtonal systems.
-------------------------	--

Available scale classes (`scale_name`):

Scale class	Character
<code>MajorScale</code>	Standard major (Ionian)
<code>MinorScale</code>	Natural minor (Aeolian)
<code>HarmonicMinorScale</code>	Minor with raised 7th
<code>MelodicMinorScale</code>	Ascending melodic minor (raised 6th & 7th)
<code>DorianScale</code>	Minor with raised 6th
<code>PhrygianScale</code>	Minor with flattened 2nd
<code>LydianScale</code>	Major with raised 4th
<code>MixolydianScale</code>	Major with flattened 7th
<code>LocrianScale</code>	Diminished-feeling mode (flattened 2nd & 5th)
<code>OctatonicScale</code>	Diminished scale — alternating whole/half steps
<code>WholeToneScale</code>	Six-note scale — all whole steps
<code>RagAsawari</code>	North Indian raga (Asavari)
<code>RagMarwa</code>	North Indian raga (Marwa)
<code>SieveScale</code>	Xenakis-style interval sieve

Example — chromatic melody snapped to melodic minor:

```
@pat(0) def ch1(p): for i in range(16): p.note(60 + i, beat=i * 0.25) p.quantize_m21("A",  
"MelodicMinorScale")
```

Example — Pythagorean tuning via Scala:

```
@pat(0) def ch1(p): p.arpeggio([60, 62, 64, 65, 67, 69, 71], step=0.25) p.quantize_m21("C",  
scala_name="pyth_12.scl")
```

Requires music21: bundled in the POMSKI installer. `pip install music21` from source.

`p.microtuning(key, scala_name, bend_range=2.0)`

Apply microtonal tuning from a Scala archive file using MIDI pitch bend. Unlike `quantize_m21`, this not only snaps notes to the nearest scale degree but also inserts a pitch bend event at each note onset that corrects the intonation by the exact cent deviation of that scale degree. A reset bend is inserted at the next note or pattern end.

This lets notes play at pitches between MIDI semitones — essential for mbira tunings, just intonation, historical temperaments, and other non-12-TET systems.

```
p.microtuning("C", "mbira_banda.scl") p.microtuning("C", "pyth_12.scl", bend_range=12)
```

Parameter	Description
<code>key</code>	Root note as a string.
<code>scala_name</code>	Filename from the Scala archive (e.g. <code>"mbira_banda.scl"</code>).
<code>bend_range</code>	Synth pitch wheel range in semitones (default <code>2.0</code>). Must match the receiving instrument's pitch bend range setting. Use <code>12</code> or <code>24</code> if your synth is configured for a wider range.

Limitation: Pitch bend is per-channel, so simultaneously playing notes share one bend value. Works best on monophonic lines or instruments where all simultaneous notes have the same tuning correction.

```
@pat(0) def ch1(p): p.brownian(start=60, steps=16) p.microtuning("C", "mbira_banda.scl") @pat(1) def ch2(p): p.euclidean(60, pulses=5) p.microtuning("C", "gamelan_udan.scl", bend_range=12)
```

`p.dropout(probability)`

Randomly remove notes.

```
p.dropout(0.2) # 20% chance of removal per note
```

`p.reverse()` — Flip the pattern backwards in time.

`p.double_time()` — Compress into first half; plays at 2× speed.

`p.half_time()` — Expand by 2×; plays at half speed. Notes outside the pattern length are removed.

p.shift(steps, grid=None)

Rotate the pattern by N grid steps.

```
p.shift(2) # shift right 2 sixteenth notes p.shift(-1) # shift left 1 step
```

p.invert(pivot=60)

Reflect all pitches around a pivot MIDI note.

```
p.invert(60) # mirror around Middle C
```

p.every(n, fn)

Apply a transformation every Nth cycle.

```
p.every(4, lambda p: p.reverse()) # reverse every 4 bars p.every(8, lambda p: p.transpose(12)) # up  
an octave every 8 bars
```

Timing & feel

p.swing(amount=57.0, grid=0.25, strength=1.0)

Apply swing feel.

```
p.swing(57) # Ableton default – gentle 16th-note shuffle p.swing(67) # triplet swing p.swing(57,  
grid=0.5) # 8th-note swing p.swing(57, strength=0.5) # half-strength blend
```

amount: 50 = straight, 57 = moderate shuffle, 67 ≈ triplet swing. Range 50–75.

p.groove(template, strength=1.0)

Apply a groove template (timing + velocity offsets).

```
from subsequence.groove import Groove groove = Groove.swing(percent=57) p.groove(groove)  
p.groove(groove, strength=0.5)
```

```
p.randomize(timing=0.03, velocity=0.0)
```

Add micro-variations to timing and/or velocity — the humanising step.

```
p.randomize() # timing only (±0.03 beats) p.randomize(timing=0.04, velocity=0.08) # both
```

Recommended ranges: `timing` 0.02–0.08 beats, `velocity` 0.05–0.15.

```
p.legato(ratio=1.0)
```

Stretch note durations to fill the gap to the next note.

```
p.legato(0.9) # 90% of the gap – slightly detached p.legato(1.0) # fully connected
```

```
p.staccato(ratio=0.5)
```

Set all note durations to a fixed proportion of a beat.

```
p.staccato(0.25) # 16th note gate (very short) p.staccato(0.5) # 8th note gate
```

```
p.velocity_shape(low=60, high=120)
```

Apply organic velocity variation using a van der Corput quasi-random sequence.

Utilities

```
p.set_length(length)
```

Dynamically change pattern length in beats.

```
if p.cycle % 4 == 3: p.set_length(2) # half-length every 4th cycle
```

```
p.param(name, default=None)
```

Read a live-tweakable parameter (set via `composition.tweak()`).

```
pitch = p.param("pitch", 60)
```

`p.signal(name)`

Read a conductor signal value at the current bar (0.0–1.0 by default).

```
density = p.signal("density")
```

LiveBridge ([Live](#))

Bidirectional bridge to Ableton Live via AbletonOSC. Silent if Live is not running; reconnects automatically every 5 seconds.

Setup: Install AbletonOSC remote script, enable it in Live Preferences → Link/Tempo/MIDI → Control Surface. Live's status bar should confirm: *"AbletonOSC: Listening for OSC on port 11000"*

State

Attribute	Type	Description
<code>live.connected</code>	<code>bool</code>	<code>True</code> if Live is reachable
<code>live.tracks</code>	<code>list[str]</code>	Track names
<code>live.scenes</code>	<code>list[str]</code>	Scene names
<code>live.clip_grid</code>	<code>list[list[int]]</code>	Clip states: 0=empty, 1=stopped, 2=playing, 3=triggered

Transport

```
live.play() # Start global transport live.stop_transport() # Stop global transport
live.set_tempo(128.0) # Set BPM in Ableton (also updates Link if active)
```

Clips & scenes

```
live.clip_play(track=0, clip=0) # Fire clip at track 0, slot 0 (0-indexed) live.clip_stop(track=0,
clip=0) # Stop a clip live.track_stop(track=0) # Stop all clips on a track live.scene_play(scene=2) #
Fire scene 2 (0-indexed)
```

Mixer

```
live.track_volume(track=0, value=0.9) # Normalised 0.0-1.0 live.track_pan(track=0, value=0.0) # -1.0
(L) to 1.0 (R), 0 = centre live.track_mute(track=1, muted=True) # Mute/unmute track
live.track_send(track=0, send=0, value=0.7) # Send level 0.0-1.0
```

Devices

```
live.device_param(track=0, device=0, param=3, value=0.7) # value is normalised 0.0-1.0 – mapped to
real range by Live's device
```

Watch (live data polling)

Poll a Live property every ~500ms and write into `composition.data`:

```
live.watch("track/0/volume") # → composition.data["live_track_0_volume"] updated every 500ms
live.watch("song/tempo", data_key="current_bpm") # → composition.data["current_bpm"]
```

Use the value inside a pattern via `p.data`:

```
@composition.pattern(channel=1, length=4) def reactive(p): vol = p.data.get("live_track_0_volume",
0.8) pitch = 60 + int(vol * 24) p.note(pitch, beat=0)
```

ClyphX Pro

live.clyphx(action) — Send any ClyphX Pro action string.

Creates a clip on a dedicated hidden MIDI track (`_POMSKI_CLYPHX`) and fires it. ClyphX Pro intercepts the clip launch and executes the action. Each concurrent call uses its own clip slot.

```
live.clyphx("BPM 120") live.clyphx("1/MUTE ON") live.clyphx("1/DEV(1) ON ; 2/ARM ON")
live.clyphx("(PSEQ) 1/MUTE ; 2/MUTE ; 3/MUTE")
```

live.clyphx_osc(address, value=1.0) — Trigger a pre-defined X-OSC address.

Faster than `clyphx()` for frequently-used actions. Bypasses AbletonOSC and sends directly to ClyphX Pro on port 7005. Requires entries in ClyphX Pro's `X-OSC.txt`.

```
# X-OSC.txt: MY_ACTION | BPM 120 live.clyphx_osc("/MY_ACTION") # trigger with value 1
live.clyphx_osc("/MUTE_1", 1) # on live.clyphx_osc("/MUTE_1", 0) # off
```

Raw OSC

```
live.send("/live/song/set/tempo", 128.0) # raw AbletonOSC message
```

DataFeeds (`feeds`)

HTTP polling helper. Fetches external APIs at regular intervals and writes results into `composition.data`.

```
# Start a feed – value lands at composition.data["feed_btc"] feeds.add("btc",
"https://api.coinbase.com/v2/prices/BTC-USD/spot", interval=10, extract=lambda r:
float(r["data"]["amount"])) # Custom headers feeds.add("gh",
"https://api.github.com/repos/python/cpython", headers={"Accept": "application/vnd.github+json"},
extract=lambda r: r["stargazers_count"]) # POST request feeds.add("custom",
"https://api.example.com/query", method="POST", body={"filter": "all"}, extract=lambda r: r["value"])
feeds.stop("btc") # stop one feed feeds.stop_all() # stop all feeds feeds # shows active feeds and
current values
```

```
feeds.add(key, url, interval=30, extract=None, headers=None, method="GET", body=None)
```

Parameter	Description
<code>key</code>	Short name. Value lands in <code>composition.data["feed_<key>"]</code>
<code>url</code>	HTTP endpoint
<code>interval</code>	Seconds between fetches (default 30)
<code>extract</code>	<code>lambda parsed_json: value</code> — stores full JSON if omitted
<code>headers</code>	Optional dict of request headers
<code>method</code>	HTTP method (default <code>"GET"</code>)
<code>body</code>	Optional request body, serialised to JSON

Use the value in a pattern:

```
@composition.pattern(channel=0, length=4) def price_melody(p): price = p.data.get("feed_btc", 0)
pitch = int(48 + min(price / 1000, 36)) p.note(pitch, beat=0)
```

Duration constants (`dur`)

```
import subsequence.constants.durations as dur
```

Constant	Beats	Musical value
<code>dur.THIRTYSECOND</code>	0.125	32nd note
<code>dur.SIXTEENTH</code>	0.25	16th note
<code>dur.DOTTED_SIXTEENTH</code>	0.375	Dotted 16th
<code>dur.TRIPLET_EIGHTH</code>	0.333	Triplet 8th
<code>dur.EIGHTH</code>	0.5	8th note
<code>dur.DOTTED_EIGHTH</code>	0.75	Dotted 8th
<code>dur.TRIPLET_QUARTER</code>	0.667	Triplet quarter
<code>dur.QUARTER</code>	1.0	Quarter note (1 beat)
<code>dur.DOTTED_QUARTER</code>	1.5	Dotted quarter
<code>dur.HALF</code>	2.0	Half note
<code>dur.DOTTED_HALF</code>	3.0	Dotted half
<code>dur.WHOLE</code>	4.0	Whole note (1 bar)

Usage:

```
# 8-step pattern at 16th-note resolution @composition.pattern(channel=1, length=8,
unit=dur.SIXTEENTH) def riff(p): p.sequence(steps=[0, 2, 5, 7], pitches=[60, 62, 64, 67])
```

GM Drum Map

Pass `GM_DRUM_MAP` as `drum_note_map` to use string names in all note placement methods.

```
from subsequence.constants.instruments.gm_drums import GM_DRUM_MAP @composition.pattern(channel=9,
length=4, drum_note_map=GM_DRUM_MAP) def drums(p): p.hit_steps("kick_1", [0, 8])
p.hit_steps("snare_1", [4, 12]) p.hit_steps("hi_hat_closed", range(16), velocity=70)
p.euclidean("ride_1", pulses=5)
```

GM name → MIDI note reference

Name	Note	Name	Note
kick_2	35	hi_hat_closed	42
kick_1	36	hi_hat_pedal	44
side_stick	37	hi_hat_open	46
snare_1	38	crash_1	49
hand_clap	39	ride_1	51
snare_2	40	chinese_cymbal	52
low_floor_tom	41	ride_bell	53
high_floor_tom	43	tambourine	54
low_tom	45	splash_cymbal	55
low_mid_tom	47	cowbell	56
high_mid_tom	48	crash_2	57
high_tom	50	ride_2	59
high_bongo	60	maracas	70
low_bongo	61	cabasa	69
mute_high_conga	62	shaker	82
open_high_conga	63	jingle_bell	83
low_conga	64	claves	75
high_timbale	65	cowbell	56
low_timbale	66	mute_triangle	80
high_agogo	67	open_triangle	81
low_agogo	68	vibraslap	58

Full list: [subsequence/constants/instruments/gm_drums.py](#) (notes 27–87).

REPL Quick Reference

Type code into the browser command box (Enter to run) or send via TCP to port [5555](#).

```
# ■■ Pattern control
composition.mute("ch3") composition.unmute("ch3") composition.running_patterns # dict of active
patterns composition.set_bpm(140) # ■■ Live tweaks
composition.tweak("bass",
pitches=[48, 52, 55, 60]) composition.clear_tweak("bass") # ■■ Ableton Live control
live.clip_play(0, 0) live.scene_play(2)
live.track_volume(0, 0.8) live.track_mute(1, True) live.set_tempo(128.0) live.clyphx("BPM 120")
live.clyphx_osc("/MY_ACTION") # ■■ Data feeds
feeds.add("btc",
"https://api.coinbase.com/v2/prices/BTC-USD/spot", extract=lambda r: float(r["data"]["amount"]))
feeds.stop("btc") feeds # list feeds and current values # ■■ Inspect state
composition.data # shared live
data dict composition.live_info() # full state snapshot live.connected live.tracks live.clip_grid
```

Command box prefixes

Prefix	Behaviour
<i>(none)</i>	Python — executed in REPL
cx:	ClyphX Pro action (e.g. cx:BPM 120)

Ports

Service	Port
Web UI (HTTP + WebSocket)	8080
REPL	5555
AbletonOSC receive	11000
AbletonOSC reply	11001
ClyphX Pro OSC	7005

MIDI channel reference

POMSKI uses 0-indexed channels (`channel=0` = MIDI ch 1).

Channel	Use
---------	-----

0–8, 10–15	Melodic instruments
9	GM percussion

Template slots [ch1](#)–[ch16](#) map to channels 0–15 respectively.